ASCEND

AI Personal Growth Companion

Product Specification & System Design Document

Version 1.0 • Prepared for handoff to AI builders / engineering teams

Prepared with Claude (Anthropic)

Table of Contents

(In Word: right-click the field below and choose "Update Field" to populate page numbers.)

1. Executive Summary

Ascend is a cross-platform (laptop + mobile), installable web application that acts as a personal growth companion. It helps the user plan daily goals and routines, tracks habits and course progress, runs a nightly planning ritual for the next day, delivers timed reminders/notifications, and closes each day with an AI-generated productivity summary and rating. It is designed to be simple, fast, and low-friction so the user actually sticks with it, while remaining fully usable offline and deployable for free.

Primary user: A single individual (self-hosted / single-tenant by default, extensible to multi-user later).

Core promise: One clear screen per moment of the day - plan at night, execute during the day, review before bed.

Deployment cost: $0/month using the free tiers described in Section 6.

2. Goals & Guiding Principles

2.1 Functional Goals (from the user's brief)

- Add daily goals, routines, and to-dos (study, workout, personal branding, custom items).
- Track habits and goals over time, with ratings and AI-generated suggestions.
- Send notifications and reminders for scheduled items.
- Every night, ask 'what are tomorrow's goals?' with timings; apply sensible default timings if none given.
- Support adding a full course/track, then get one topic/video/point suggested per day, in sequence.
- One hour before bedtime, generate a summary of the day and a productivity rating.
- Track everything the user does inside the app over time (history + analytics).
- Work online (deployed, free) and offline (locally on phone or laptop) without losing functionality.
- Stay simple and pleasant enough to build a long-term daily habit around.

2.2 Design Principles Added for This Spec

- Local-first, cloud-optional: the app must be 100% usable with zero internet connection; sync is a bonus layer, not a dependency.
- One primary screen ("Today"): everything else is one tap away. No feature should require more than 2 taps from Today.
- Progressive disclosure: courses, analytics, and settings are secondary screens - never shown by default unless relevant.
- Every AI feature degrades gracefully: if there's no internet/API access, the app falls back to simple rule-based logic (e.g., a template summary instead of an AI one) rather than breaking.
- Habit formation over feature richness: streaks, gentle nudges, and a forgiving design (missing a day doesn't reset everything to zero) so the user doesn't quit after one bad day.
- Single source of truth for time: all reminders, the nightly ritual, and the pre-bed summary are computed from one user-defined daily schedule (wake time, sleep time, timezone).

2.3 A Note on Scope: Simplicity vs. Richness

This spec was reviewed against two external drafts. One proposed a much larger vision - a full 'life operating system' with 20+ subsystems (a Notion-style knowledge base, multi-persona AI mentors, project management,

decision journals, and more). Many of those ideas are genuinely good, but building all of them first is the most common way habit apps never ship or never get used - it directly contradicts the founding requirement that this app must stay simple enough to stick to.

The resolution used throughout this document: lightweight ideas that add depth without adding screens or new subsystems are folded directly into the relevant feature below (marked inline). Larger, standalone-product-sized ideas are preserved in Section 14 (Future Vision) so nothing is lost, but are explicitly kept out of the MVP and the 5-screen core experience.

3. Primary User Scenario (Day in the Life)

When	What happens
10:30 PM (1 hr before bed)	App pushes a notification: "How did today go?" Opens the Daily Review screen -> shows checklist of what was planned vs. done, auto-drafted AI summary + productivity score (0-100), user can edit/confirm.
10:40 PM	App then asks the Nightly Planning question: "What are your goals for tomorrow?" User types or picks from templates (Study, Workout, Personal Branding, Custom) and optionally sets times. Untimed items get smart defaults (Section 5.3).
Next day, each reminder time	Push/local notification fires: "Time to: Workout (7:00 AM)". Tapping it opens the task with Done / Snooze / Skip actions.
During the day	If the user has an active Course/Track (e.g., "Machine Learning Specialization"), the Today screen shows one card: "Today's lesson: Week 3 - Gradient Descent" with a Mark Complete button. Completing it auto-queues the next item for tomorrow.
Anytime	User can open Ascend and add a new goal, habit, task, or course; view streaks and analytics; or edit tonight's plan.

4. Feature Specification

4.1 Goals, Routines & Tasks

A unified 'Item' model covers goals, routines, and one-off tasks, distinguished only by a type + recurrence field (see Section 7 data model). This avoids building three separate systems.

- Create item with: title, category (Study / Workout / Personal Branding / Custom / ...), type (one-off task, daily routine, long-term goal), optional time, optional duration, optional notes.
- Recurring routines auto-regenerate each day (e.g., "Morning workout" reappears every day until paused).
- Long-term goals can have linked sub-tasks/milestones and a target date; progress rolls up from sub-tasks.
- Quick-add bar on the Today screen: natural free-text entry, e.g. 'Workout 7am' or 'Read 30 min', parsed into title + time.
- Custom categories/tags so the system is never a hard limit on what the user can track.
- Optional Mission linkage: a goal can point to a short user-written 'life mission' statement (e.g., 'Become a strong AI engineer'). When present, the app occasionally surfaces it above today's plan - a one-line 'why' reminder, not a separate tracking system.

4.2 Habit & Goal Tracking + Ratings

- Every recurring routine/habit gets a streak counter (current streak, best streak) and a completion history (calendar heatmap, GitHub-contributions style).
- Daily 'consistency score' per category (e.g., Study 80%, Workout 60% this week) computed from completed vs. planned items.
- Weekly and monthly rollups with trend arrows (improving / declining) to feed the AI suggestion engine.
- AI Suggestions Engine reviews the last 7-14 days and produces short, specific nudges, e.g. 'Workout completion dropped from 90% to 40% this week - try moving it to mornings, your best completion time slot.' Suggestions are template-based fallback + LLM-enhanced when online.
- Habit stacking: when creating a habit, the user can optionally anchor it to an existing routine, e.g. 'After [Morning Coffee], I will [Review 1 GitHub Repo]' - shown as a linked pair on the Today screen so the anchor habit visually cues the new one.
- Automatic scale-down suggestion: if a habit is missed 3 consecutive days, the app proactively suggests a smaller version of it (e.g., 'Instead of a 1-hour workout, try 15 minutes today') rather than letting the streak break outright - lowering the bar back to zero is treated as a last resort, not a failure.
- Anti-burnout signal: a simple heuristic (many consecutive high-load days, declining completion rate, and - if the user opts into the energy check-in in 4.10 - low logged energy/sleep) triggers a one-time suggestion to plan a deliberately lighter day, rather than silently letting the user grind into a slump.

4.3 Nightly Planning Ritual

- Triggered automatically by a local notification at a user-configurable 'planning time' (default: 1.5 hours before bedtime).
- Conversational, single-screen flow: 'What are your goals for tomorrow?' -> free text or quick-pick chips per category -> optional time per item.
- If no time is given, the Default Timing Engine assigns one (Section 4.5).

- Ends by showing the full draft plan for tomorrow with an 'Looks good' / edit option.
- Daily Focus pick: as part of the same flow, the user marks 1-3 of tomorrow's items as the 'Top' priority. The Today screen visually leads with these - everything else is present but secondary, so a busy day never feels like 17 equally-urgent demands.

4.4 Course / Track Sequential Learning

- User can create a 'Track' (e.g., a course) either manually (paste a list of topics/videos/chapters) or by pasting a syllabus which is auto-split into ordered items.
- Each day, the Today screen shows exactly one active item from each active track: 'Today's lesson'.
- Marking it complete advances a pointer to the next item for the following day - never shows more than one at a time, to avoid overwhelm.
- If a day is missed, the track simply pauses at the current item - it never queues up a backlog of missed lessons to 'catch up on'. The next day still shows exactly one item, so falling behind never compounds into overwhelm.
- Supports pausing/resuming a track, reordering items, and marking items complete out of order if the user studies ahead.
- Progress bar per track (e.g., '12 / 48 lessons - 25%').

4.5 Default Timing Engine

When the user adds an item without a specific time, the system assigns one from configurable defaults, editable in Settings:

Category	Default time	Default duration
Study / Course lesson	9:00 AM	45 min
Workout	7:00 AM	45 min
Personal branding (content, posting, networking)	6:00 PM	30 min
General task (no category)	Next free slot today	30 min
Custom category	User's last-used time for that category, else 12:00 PM	30 min

4.6 Notifications & Reminders

- Per-item reminder at its scheduled time (push notification when online/installed; local notification via Service Worker when offline).
- Nightly planning prompt at the configured planning time.
- Pre-bed summary prompt exactly 60 minutes before the configured bedtime.
- Optional gentle re-nudge if a task is still not marked done 30-60 minutes after its time.
- Snooze (10/30/60 min) and Skip actions directly from the notification.
- Smart rescheduling: if a timed item is missed, the app proposes the next realistic free slot today (computed from the rest of the day's schedule) with a one-tap 'Move it' action, instead of leaving the user to manually replan.
- All notification times are computed client-side from the user's local timezone and schedule - see Section 8 for implementation.

4.7 Pre-Bed Daily Summary & Productivity Rating

- Fires 1 hour before bedtime (configurable).
- Aggregates: items completed vs planned, time spent (if tracked), streaks affected, course progress made.
- Produces a Productivity Score (0-100) using a transparent weighted formula (Section 9), so the user can trust and understand the number, not just an opaque AI guess.
- When online, an LLM call turns the raw stats into 2-4 sentences of specific, encouraging, non-generic feedback plus one concrete suggestion for tomorrow. Offline, a template-based summary is generated from the same stats.
- Optional reflection prompt: one short rotating question (e.g., 'What challenged you today?' or 'What's one thing future-you would thank you for?') the user can answer in a sentence or skip entirely - stored alongside the log, never required.
- Summary and score are saved permanently to the history log for long-term trend analysis.

4.8 Analytics Dashboard

- Calendar heatmap of daily completion / productivity score.
- Category breakdown (pie/bar: time and consistency per category).
- Streak leaderboard across the user's own habits.
- Course progress overview across all active tracks.
- Life Balance Wheel: a radar chart of consistency-per-category (Study, Workout, Personal Branding, etc.) - built entirely from data already tracked in Section 4.2, just visualized differently to reveal imbalance at a glance.
- Achievement Timeline: a simple chronological feed auto-generated from completed goals, finished tracks, and streak milestones - 'Finished IBM Data Analytics - Sept 2026' - reusing existing daily_logs/track data, not a new subsystem.
- Exportable history (CSV/JSON) so the user always owns their data.

4.9 Gamification (added value, optional toggle)

- Streaks and a simple XP/level system tied to consistency, not raw volume, so quality days count more than busy-but-shallow days.
- Milestone badges (7-day streak, 30-day streak, course completed) - purely cosmetic, never punitive.
- Can be fully disabled in Settings for users who find gamification distracting.

4.10 Optional Energy Check-in

- A single one-tap scale (e.g., 1-5) for energy/mood, offered once in the morning and once at the pre-bed review - never a full mood-journaling system.
- Purely optional and skippable; when present over enough days, it feeds the Anti-Burnout Signal (4.2) and lets the AI suggestion engine notice patterns like 'your best-completed study sessions happen when you log higher morning energy.'
- No separate screen - it's a single inline control on the Today and Daily Review screens.

5. UX / UI Design Principles

- Screens are limited to five: Today, Plan Tonight, Tracks/Courses, Analytics, Settings. Everything else is a modal/sheet on top of these.
- The Today screen shows, top to bottom: greeting + date, pre-bed/planning prompt banner (only when due), today's schedule as a simple timeline list with checkboxes, and the single active 'Today's lesson' card per track.
- Mobile-first responsive layout (single column on phone, two-column on laptop/tablet) built with a utility-first CSS approach for consistency across breakpoints.
- Dark mode by default (reduces friction for nightly use), light mode optional.
- Missed items turn neutral grey, never red/alarming - the visual language never punishes; guilt-driven UI is a well-documented cause of habit-tracker abandonment.
- Desktop-only command palette (Cmd/Ctrl+K) for instantly adding a task or jumping between Today / Tracks / Analytics without touching the mouse - keeps power users fast without adding visual complexity for everyone else.
- Micro-interactions only where they reinforce habit (checkbox tick animation, streak flame icon) - no clutter, no ads, no infinite feeds.
- Every AI-generated text (summary, suggestion) is clearly editable/dismissible - the user is always in control, the AI assists but never blocks.

6. Recommended Technology Stack (Free, Cross-Platform, Offline-Capable)

The single architecture that satisfies 'laptop + mobile', 'free deployment', and 'usable fully offline' at once is a Progressive Web App (PWA) with a local-first database. A PWA installs like a native app on Android, iOS (Add to Home Screen), Windows, and macOS from one codebase, works fully offline via a Service Worker, and costs nothing to host on static free tiers.

Layer	Recommended choice	Why / free tier notes
Frontend framework	React + Vite + TypeScript	Fast dev/build, huge ecosystem, tiny bundle with Vite.
Styling	Tailwind CSS	Consistent, fast, mobile-first utility classes.
App shell / installability	PWA (Web App Manifest + Service Worker via Vite PWA plugin)	Installable on phone & laptop; enables offline caching and background notification scheduling.
Local/offline storage	IndexedDB via Dexie.js	Structured, queryable local database that works fully offline; source of truth on-device.
Cloud sync & backend (optional)	Supabase (Postgres + Auth + Realtime)	Generous free tier; used only to sync/back up data across devices when online - app must work without it.
Auth	Supabase Auth (email/magic link) or fully local single-user mode	Optional - a single-user local mode needs no login at all.
Notifications	Web Push API + Notification API + Service Worker timers; optional OS-level reminders via installed PWA	No paid push service required for a single user; Supabase Edge Functions can trigger push if server-side timing is needed.
AI features	Anthropic Claude API (nightly planning parsing, suggestions, daily summary)	Pay-as-you-go, very low volume for one user (a few calls/day); falls back to rule-based logic offline. For a strictly $0 AI budget, Google Gemini's free API tier is a viable drop-in alternative behind the same adapter (Section 10.2).
Hosting	Vercel or Netlify (free tier) or GitHub Pages	Static PWA deploy, HTTPS + custom domain included, auto-deploy from GitHub.
Local-only fallback	Same app served via `npm run build` + any static file server, or opened directly as an installed PWA	No deployment required at all if the user only wants it on their own devices.

6.1 Why not a native app (React Native/Flutter)?

Native apps would require app-store accounts, review processes, and separate builds per OS - all in tension with 'free' and 'simple to deploy'. A PWA reaches laptop and mobile from one codebase, is free to host, and supports offline use and local notifications well enough for this use case. Native app support can be revisited later as a Phase 3 item if desired.

6.2 Optional Server-Side Reliability Layer (Phase 3)

The client-only design above has one real gap: course-track advancement and time-based notifications only get computed when the app is actually opened. For a single user this is a minor limitation (opening the app once a day still works fine), but if fully server-reliable, app-closed notifications and course drip are wanted, add a small serverless layer rather than a full separate backend service:

- A scheduled serverless function (Supabase Edge Function cron, or a Vercel Cron job) runs once daily to advance each active track's pointer and compute which notifications are due - independent of whether the client app is open.
- This keeps the architecture serverless and free-tier friendly, rather than introducing a second always-on service (e.g., a standalone FastAPI server) that would need its own free-tier host and uptime management for very little added benefit at single-user scale.
- Recommended as a Phase 3 enhancement, only after the local-first MVP (Section 12) is working end-to-end.

7. Data Model

All entities live in the local IndexedDB (Dexie) database and, optionally, mirror to a Postgres schema in Supabase with the same shape for sync/backup.

7.1 Core Tables

Table	Key fields	Notes
users	id, name, timezone, wake_time, sleep_time, planning_time, bedtime_summary_time	Single row in local-only mode.
missions	id, user_id, statement, created_at	Optional one-line 'why' a goal can link to (Section 4.1). Not a separate tracking system.
items	id, user_id, title, category, type (task/routine/goal), recurrence_rule, scheduled_time, duration_min, est_minutes, actual_minutes, is_top_focus, status (pending/done/skipped), goal_id (nullable), created_at	Unified table for tasks, routines, and goal sub-tasks. est/actual minutes feed smarter future estimates; is_top_focus flags a Daily Focus pick (4.3).
goals	id, user_id, title, category, target_date, progress_pct, mission_id (nullable)	Long-term goals; linked items roll up progress; optional mission link.
habits	id, user_id, item_template_id, current_streak, best_streak, last_completed_date	Derived/cached streak data for recurring items.
tracks	id, user_id, title, source_type (manual/pasted-syllabus), current_index, status (active/paused/done)	A course/learning track.
track_items	id, track_id, order_index, title, notes/url, status (pending/done)	Ordered lessons/topics/videos within a track.
daily_logs	id, user_id, date, planned_items_json, completed_count, planned_count, productivity_score, ai_summary_text, reflection_text (nullable), energy_am (nullable), energy_pm (nullable)	One row per day; the permanent history record.
notifications	id, user_id, item_id (nullable), fire_at, type (reminder/planning/summary), status (scheduled/fired/dismissed)	Local schedule of pending notifications.
settings	user_id, defaults_json (per-category default times), gamification_enabled, ai_tone_preference, theme	User preferences, including how the AI should phrase feedback (direct vs. encouraging).

7.2 Example Item Record (JSON)


```
{   "id": "itm_2b1e",   "title": "Workout",   "category": "Workout",   "type": "routine",
"recurrence_rule": "daily",   "scheduled_time": "07:00",   "duration_min": 45,   "status":
"pending",   "goal_id": null,   "created_at": "2026-07-04T22:41:00+05:00" }
```

8. Notification & Reminder System - Implementation Design

- On app load/install, register a Service Worker; request Notification permission once, with a clear explanation screen first (never on cold start silently).
- Maintain a local 'notifications' table of upcoming fire times, recomputed whenever items are added/edited or at midnight rollover.
- Use the Periodic Background Sync API / Service Worker timers where supported (Android Chrome, desktop) to fire notifications even when the tab is closed, as long as the PWA is installed.
- iOS Safari has limited background notification support even for installed PWAs; document this limitation and provide an in-app 'Today' badge plus an optional daily digest email/push via Supabase Edge Function as a fallback for iOS users.
- Notification payload includes item id and 3 quick actions: Done, Snooze 10m, Skip - handled by the Service Worker's notificationclick event, writing directly to IndexedDB so it works offline.
- The Nightly Planning and Pre-Bed Summary notifications are scheduled from `bedtime - 90min` and `bedtime - 60min` respectively, recalculated whenever the user changes their bedtime in Settings.

9. Productivity Score Formula (Transparent, Explainable)

A simple, explainable weighted formula avoids an opaque 'AI decided' feeling and lets the user trust the number. Each item carries a priority weight (default 1, higher for user-marked 'important' items), so a completed high-priority goal counts more than a trivial checkbox, and an unrescheduled high-priority miss costs more than a low-priority one:

```
raw_score = SUM(w_i * c_i for each planned item i)   // w = item priority weight, c = 1 if
done else 0           + H                                        // habit streak bonus,
capped contribution           - D                                          // penalty:
unrescheduled high-priority misses  score = normalize(raw_score against SUM(w_i)) -> scaled
to 0-100, floored at 0
```

A missed item only becomes a 'D' penalty if it was high-priority AND the user never rescheduled or skipped it intentionally - a consciously rescheduled task is not punished, only an ignored one. This keeps the score fair rather than mechanically harsh.

The LLM only turns this number and its inputs into friendly written feedback - it never invents the score itself, keeping the metric consistent and auditable day to day.

10. AI Integration Design

10.1 Where AI Is Used

Feature	AI's job	Offline fallback
Nightly planning	Parse free-text goals into structured items with title/category/time	Simple keyword/regex parser + manual chip picker
Suggestions engine	Turn 7-14 day stats into a specific, actionable nudge	Static rule-based templates (e.g., 'X dropped below 50% - consider changing the time')
Daily summary	Turn the day's stats into a short encouraging, specific write-up	Template sentence built from the same stats (e.g., '6/8 tasks done, 75% - your Workout streak is now 5 days.')
Syllabus splitting	Convert a pasted course outline into an ordered list of track items	Manual line-by-line paste, one item per line

10.2 Prompting Approach

All AI calls are short, single-purpose, and stateless: they receive only the structured stats/text needed for that task (never the whole history) and are asked to return strict JSON where the app needs structured data (e.g., parsed goals) or a short plain-text passage for the daily summary. This keeps calls cheap, fast, and predictable.

The LLM call sits behind a single adapter function per feature (e.g., getDailySummary(stats)), so the underlying provider can be swapped without touching app logic - Claude API by default for quality, or Google Gemini's free tier as a drop-in if the goal is a strictly $0 AI budget.

Consistency of voice: every AI prompt includes the user's stored tone preference (Section 7, settings.ai_tone_preference - e.g., 'direct' vs. 'encouraging') so feedback feels like the same companion over time rather than a generic model each time. This is a single prompt-template parameter, not a separate personality engine.

11. Non-Functional Requirements

Category	Requirement
Performance	Today screen must render in under 1 second from local data; no feature should block on network.
Offline	Full CRUD on goals/habits/tracks and the nightly/summary flows must work with zero network connectivity.
Privacy	All personal data stays on-device by default; cloud sync is opt-in and encrypted in transit (HTTPS) and at rest (Supabase default).
Portability	User can export all data as JSON/CSV at any time; no lock-in.
Reliability	Missing a day never deletes history or resets long-term stats to zero - only the current streak resets.
Cost	$0 baseline hosting/storage; only variable cost is optional low-volume AI API usage.
Accessibility	Sufficient color contrast in both themes; all interactive elements keyboard- and screen-reader-accessible.

12. MVP & Roadmap

Phase 1 - MVP (2-3 weeks solo build)

- Today screen with items (add/edit/complete), categories, and default timing engine.
- Local storage only (IndexedDB), installable PWA, local notifications.
- Nightly planning flow and pre-bed summary using the rule-based (non-AI) fallback logic first.
- Basic streaks and a simple weekly view.

Phase 2 - AI Layer

- Wire the Anthropic Claude API into planning parsing, suggestions, and daily summaries.
- Add the Analytics dashboard (heatmap, category breakdown).
- Add course/track sequential learning feature.

Phase 3 - Sync & Polish

- Optional Supabase account + cross-device sync.
- Gamification layer (streak badges, levels) as a togglable feature.
- iOS notification fallback (digest push/email) if needed.
- Optional serverless cron layer (Section 6.2) for app-closed course drip and notification reliability.
- Desktop command palette (Cmd/Ctrl+K) for fast task entry and navigation.
- Weekly AI check-in: a Sunday variant of the same summary engine (Section 4.7) reviewing the week instead of the day - reuses existing logic, no new AI system.

13. Future Vision - Optional Expansion Modules (Post-MVP)

The ideas below (surfaced during spec review) are each substantial enough to be their own product. They are intentionally excluded from the MVP and the 5-screen core experience (Section 5) so the app stays simple enough to actually stick with. If Ascend is still in daily use after several months, these are the natural next layer - each could become a single new area of the app rather than being bolted onto Today.

Module	One-line concept
Second Brain	A searchable Notion/Obsidian-style vault for notes, ideas, quotes, and resources, linked to goals and tracks.
AI Mentor Personas	Switchable coaching modes (career, fitness, study, business) reasoning over the same underlying data.
Project Management	Multi-task projects with milestones, files, and deadlines, for work that outgrows a single goal/track.
Decision Journal	Log a decision, the reasoning, and the prediction; AI reviews outcomes later for patterns.
Personal AI Profile	A continuously-updated model of strengths, best study hours, and common distractions, feeding smarter suggestions.
Challenge Mode	Time-boxed challenges (30-day workout, 100-day coding streak) as an optional motivational format.
Context Awareness	Auto-adjusting schedule defaults around exams, travel, religious observances, or vacation periods.
Opportunity Detector	Suggests concrete next steps (courses, competitions, portfolio ideas) based on recently completed skills.
Focus Music	One-tap ambient sound (lo-fi, rain, white noise) alongside a focus/deep-work timer.

None of these require changes to the core data model in Section 7 to add later - they would layer on top of the existing items/goals/tracks tables rather than replacing them, which is why the MVP is safe to build now without over-engineering for a future that may or may not be wanted.

14. Notes for the Building AI / Engineering Team

- Build local-first: get IndexedDB (Dexie) CRUD and the Today screen fully working before touching any cloud or AI service.
- Keep the unified 'items' model (Section 7.1) rather than separate tables per feature - it is what keeps the app simple to extend.
- The Default Timing Engine, Productivity Score formula, and notification scheduling should be pure, testable functions, independent of UI, so they can be unit-tested and later reused if a native app is ever built.
- Treat every AI call as optional/replaceable - wrap each in a small adapter (e.g., `getDailySummary(stats)`) with the rule-based fallback as the default, and the Claude API call as an enhancement when a network + API key are available.
- Ship Phase 1 (Section 12) completely before adding AI or sync, so the user already has a usable, offline habit-tracking app at every step.

14.1 Environment & Credentials Practice

- Use an isolated environment appropriate to the stack: a Node version manager (nvm) + a project-local node_modules for the frontend; if any Python tooling is used anywhere (e.g., a data/notebook helper script), create a virtual environment (python -m venv) rather than installing packages globally.

- Never hardcode API keys, database URLs, or any credential directly in source code. All secrets go in a local .env file (added to .gitignore) and, in production, in the hosting provider's environment variable settings (Vercel/Netlify/Supabase dashboards).
- Whenever the build requires something only the account owner can provide - an Anthropic/Gemini API key, a Supabase project URL + anon key, a push notification VAPID key, or any other external credential - stop and ask Muhammad Faheem directly for it rather than inventing a placeholder and moving on. The same applies if a choice needs a real decision (e.g., which free tier, which domain name).
- Commit a .env.example file (with variable names only, no real values) so it's always clear which credentials the app needs without exposing any of them.

15. Deployment Guide - Step by Step (For Muhammad Faheem)

Once the app is built, here is how to get it running for free, both online and offline, on your laptop and phone.

15.1 One-Time Setup

- Create a free GitHub account (if you don't have one) and push the finished project code to a new GitHub repository.
- Create a free Vercel account (or Netlify) and sign in with GitHub.
- If using cloud sync: create a free Supabase account and a new project - Supabase will give you a Project URL and an anon/public API key.
- If using AI features: get an Anthropic API key from console.anthropic.com (or a Google Gemini API key from Google AI Studio for the $0 option).

15.2 Deploying the App Online (Free)

- In Vercel, click 'Add New Project' and select your GitHub repository - Vercel auto-detects the framework and builds it.
- Before the first deploy, add your credentials under Project Settings -> Environment Variables (e.g., ANTHROPIC_API_KEY, SUPABASE_URL, SUPABASE_ANON_KEY). Never put these in the code itself.
- Click Deploy. Vercel gives you a free https:// URL (e.g., ascend-yourname.vercel.app) - this is your live app.
- Every time you push new code to GitHub, Vercel automatically redeploys the updated version - no manual redeploy step needed.

15.3 Installing on Your Phone

- Open your Vercel URL in Chrome (Android) or Safari (iPhone).
- Android/Chrome: tap the browser menu -> 'Add to Home screen' / 'Install app'. It now behaves like a native app icon, works offline, and can show notifications (with the iOS limitation noted in Section 8).
- iPhone/Safari: tap the Share icon -> 'Add to Home Screen'.

15.4 Installing on Your Laptop

- Open your Vercel URL in Chrome or Edge.
- Click the install icon in the address bar (or browser menu -> 'Install Ascend...'). It now opens in its own window like a desktop app, and keeps working offline.

15.5 Fully Local / Offline-Only Option (No Deployment at All)

- If you don't want to deploy anywhere, you can run the app entirely on your own machine: build it (the exact command will be provided by whoever builds the app, typically something like npm run build), then open the built app in your browser or serve it with a simple local static server.
- In this mode there is no cloud sync and no online AI calls (unless you're connected to the internet at that moment) - but every core feature (planning, tracking, notifications, summaries) still works using the offline fallbacks described throughout this document.

15.6 Updating the App Later

- Ask your AI builder / engineer for the specific change.
- Once the updated code is pushed to your GitHub repository, Vercel redeploys automatically - your installed phone and laptop apps will pick up the update the next time you open them.

That's it - one deploy, free hosting, installable everywhere, and it never stops working just because the internet does.